Python Notes: Math in Python is Easy!

Video: *Math in Python is easy!*

Video URL: *https://www.youtube.com/watch?v=jc7TBgMS_kw*

Topics covered: Arithmetic operators, augmented assignment, built-in math functions, the `math` module, and practical programming exercises.

1. Introduction to Mathematics in Python

Python provides arithmetic operators, built-in functions, and a dedicated `math` module to perform mathematical calculations.

These features are useful in everyday programming tasks, such as calculating prices, finding the area of a shape, determining whether a number is even or odd, and solving mathematical problems.

Three main areas

1. **Arithmetic operators:** Perform calculations such as addition, subtraction, multiplication, and division.
2. **Built-in mathematical functions:** Perform operations such as rounding, finding absolute values, calculating powers, and identifying maximum or minimum values.
3. **The `math` module:** Provides mathematical constants and functions, including pi, square roots, ceiling, and floor.

---

2. Basic Arithmetic Operators

Arithmetic operators perform mathematical calculations on numbers.

Operator	Name	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`5 - 3`	`2`
`*`	Multiplication	`5 * 3`	`15`
`/`	Division	`10 / 2`	`5.0`
`**`	Exponentiation	`5 ** 2`	`25`
`%`	Modulus	`10 % 3`	`1`
`//`	Floor division	`10 // 3`	`3`

IMPORTANT

Note: The video focuses mainly on addition, subtraction, multiplication, division, exponentiation, and modulus. Floor division is also included here as a useful related operator.

2.1 Addition (`+`)

The addition operator adds two numbers.

```
a = 10
b = 5
result = a + b
print(result)
```

Output:

```
15
```

2.2 Subtraction (-)

The subtraction operator subtracts one number from another.

```
a = 10
b = 15
result = a - b
print(result)
```

Output:

```
-5
```

A negative result means the second number is greater than the first.

2.3 Multiplication (*)

The multiplication operator multiplies two numbers.

```
friends = 5
friends = friends * 3
print(friends)
```

Output:

```
15
```

2.4 Division (/)

The division operator divides one number by another.

```
a = 5
result = a / 2
print(result)
```

Output:

```
2.5
```

IMPORTANT PYTHON RULE

In Python 3, the `/` operator returns a floating-point result, even when the division is exact.

```
print(10 / 2)
```

Output:

```
5.0
```

2.5 Exponentiation (**)

The exponentiation operator raises a number to a specified power.

Syntax: `base ** exponent`

```
print(5 ** 2)  # 25
print(2 ** 3)  # 8
print(10 ** 0) # 1
```

Explanation:

- `5 ** 2` means $5 \times 5 = 25$.
- `2 ** 3` means $2 \times 2 \times 2 = 8$.
- `10 ** 0` equals `1`.

This operator is particularly useful for calculating squares and cubes.

2.6 Modulus (`%`)

The modulus operator returns the **remainder** after division.

Syntax: `number % divisor`

```
print(10 % 3)  # 1
print(10 % 2)  # 0
print(17 % 5)  # 2
```

Explanation:

- `10 % 3` gives `1` because 3 goes into 10 three times, with 1 left over.
- `10 % 2` gives `0` because 10 is divisible by 2.
- `17 % 5` gives `2` because 5 goes into 17 three times, with 2 left over.

Application: Checking even and odd numbers

The modulus operator is commonly used to determine whether an integer is even or odd.

```
number = 7
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output:

```
Odd
```

RULE TO REMEMBER

- If `number % 2 == 0`, the integer is even.
- If `number % 2 != 0`, the integer is odd.

For integer inputs, this is a simple and reliable way to check parity.

2.7 Floor Division (`//`)

Floor division divides two numbers and rounds the quotient down to the nearest integer value.

```
print(10 // 3)  # 3
print(15 // 4)  # 3
print(-10 // 3) # -4
```

Notice that floor division with negative numbers rounds toward negative infinity rather than simply removing the decimal part.

3. Augmented Assignment Operators

Augmented assignment operators combine an arithmetic operation and assignment in a shorter statement.

For example, instead of writing:

```
friends = friends + 1
```

You can write:

```
friends += 1
```

Both statements increase the value of `friends` by one.

3.1 Common augmented assignment operators

Operator	Equivalent expression	Meaning
`+=`	`x = x + value`	Add and assign
`-=`	`x = x - value`	Subtract and assign
`*=`	`x = x * value`	Multiply and assign
`/=`	`x = x / value`	Divide and assign
`**=`	`x = x ** value`	Raise to a power and assign
`%=`	`x = x % value`	Calculate remainder and assign
`//=`	`x = x // value`	Floor-divide and assign

3.2 Examples

Addition assignment

```
friends = 0
friends += 1
print(friends)
```

Output: `1`

Subtraction assignment

```
friends = 10
friends -= 2
print(friends)
```

Output: `8`

Multiplication assignment

```
friends = 5
friends *= 3
print(friends)
```

Output: `15`

Division assignment

```
number = 10
number /= 2
print(number)
```

Output: `5.0`

Exponentiation assignment

```
number = 5
number **= 2
print(number)
```

Output: `25`

Modulus assignment

```
number = 10
number %= 3
print(number)
```

Output: `1`

IMPORTANT

Augmented assignment updates the variable. Choose the operator according to the operation you need to perform.

4. Built-in Mathematical Functions

Python provides useful mathematical functions without requiring you to import an additional module.

The video covers five important functions:

- round()
- abs()
- pow()
- max()
- min()

4.1 The round() Function

The round() function rounds a number to a specified number of decimal places.

Syntax: round(number, ndigits)

- number : The value to round.
- ndigits : The number of decimal places to retain. This argument is optional.

Example 1: Round to the nearest integer

```
x = 3.14
result = round(x)
print(result)
```

Output: 3

Example 2: Round to two decimal places

```
number = 12.3456
result = round(number, 2)
print(result)
```

Output: 12.35

Example 3: Round to three decimal places

```
print(round(3.141592, 3))
```

Output: `3.142`

IMPORTANT

Python's `round()` does not always behave like the everyday rule of rounding every exact halfway value upward. For exact halfway cases, Python generally uses *round half to even*. For example:

```
print(round(2.5)) # 2
print(round(3.5)) # 4
```

Also, `round()` returns a rounded numeric value; it does not guarantee that a particular number of trailing zeros will be displayed. For display formatting, an f-string such as `f"{value:.2f}"` can be more suitable.

4.2 The `abs()` Function

The `abs()` function returns the absolute value of a number. Absolute value represents a number's distance from zero, without considering its sign.

Syntax: `abs(number)`

```
print(abs(-4))   # 4
print(abs(4))    # 4
print(abs(-7.5)) # 7.5
print(abs(0))    # 0
```

Key concept: Absolute values are never negative.

4.3 The `pow()` Function

The `pow()` function raises a base number to a specified exponent.

Syntax: `pow(base, exponent)`

```
print(pow(4, 3)) # 64
print(pow(5, 2)) # 25
print(pow(2, 4)) # 16
```

For example, `pow(4, 3)` means $4^3 = 4 \times 4 \times 4 = 64$.

You can achieve the same basic power calculation with the `**` operator:

```
print(4 ** 3) # 64
```

Remember: `pow(4, 3)` and `4 ** 3` both calculate the same power in these examples.

4.4 The `max()` Function

The `max()` function returns the largest value among the supplied values.

Syntax: `max(value1, value2, value3, ...)`

```
x = 3.14
y = 4
z = 5
result = max(x, y, z)
print(result) # 5
```

```
print(max(10, 20, 5))      # 20
print(max(-4, -1, -9))     # -1
print(max(2.5, 8.7, 6.3))  # 8.7
```

4.5 The `min()` Function

The `min()` function returns the smallest value among the supplied values.

Syntax: `min(value1, value2, value3, ...)`

```
print(min(10, 20, 5))  # 5
print(min(-4, -1, -9)) # -9
```

Function	Purpose	Example	Result
`max()`	Largest value	`max(3, 7, 2)`	`7`
`min()`	Smallest value	`min(3, 7, 2)`	`2`

5. The Python `math` Module

Python's `math` module provides additional mathematical constants and functions. To use it, write:

```
import math
```

Access constants and functions using dot notation: `math.function_name()` or `math.pi`.

5.1 The Constant `math.pi`

Pi (π) is used in calculations involving circles. $\pi \approx 3.1415926535897793$.

```
import math
print(math.pi)
```

Output: `3.141592653589793`

5.2 The Constant `math.e`

Euler's number (e) is important in exponential growth, calculus, and scientific calculations. $e \approx 2.718281828459045$.

```
import math
print(math.e)
```

Output: `2.718281828459045`

5.3 The `math.sqrt()` Function

Returns the square root of a non-negative number.

```
import math
print(math.sqrt(9))  # 3.0
print(math.sqrt(16)) # 4.0
print(math.sqrt(2))  # 1.4142...
```

IMPORTANT

`math.sqrt()` does not accept negative real numbers. Passing a negative number raises a `ValueError`.

5.4 The `math.ceil()` Function

Returns the smallest integer greater than or equal to a number.

```
print(math.ceil(9.1))  # 10
print(math.ceil(9.9))  # 10
print(math.ceil(-9.1)) # -9
```

5.5 The `math.floor()` Function

Returns the greatest integer less than or equal to a number.

```
print(math.floor(9.9))  # 9
print(math.floor(9.1))  # 9
print(math.floor(-9.1)) # -10
```

Feature	math.ceil()	math.floor()
Meaning	Smallest int \(\ge\) value	Greatest int \(\le\) value
9.1	10	9
9.9	10	9
-9.1	-9	-10

6. Quick Reference: Built-in vs. math Module

Function / Constant	Requires import math ?	Purpose
round()	No	Rounds a number
abs()	No	Returns absolute value
pow()	No	Calculates a power
max() / min()	No	Finds largest / smallest value
math.pi / math.e	Yes	Provides pi / Euler's number
math.sqrt()	Yes	Square root
math.ceil() / floor()	Yes	Rounds up / down

7-9. Practical Exercises & Revision

Exercise 1: Circumference of a Circle

Circumference $C = 2\pi r$

For radius 10.5 cm: C = 2 * math.pi * 10.5 ≈ 65.97 cm

```
import math
radius = float(input("Enter radius: "))
circumference = round(2 * math.pi * radius, 2)
print(f"Circumference: {circumference} cm")
```

Exercise 2: Area of a Circle

Area $A = \pi r^2$

For radius 10.5 cm: A = math.pi * pow(10.5, 2) ≈ 346.36 cm²

```
import math
radius = float(input("Enter radius: "))
area = round(math.pi * pow(radius, 2), 2)
print(f"Area: {area} cm²")
```

Exercise 3: Hypotenuse of a Right Triangle

Pythagorean Theorem: $c = \sqrt{a^2 + b^2}$

For a=3, b=4: c = math.sqrt(3**2 + 4**2) = 5.0

```
import math
a = float(input("Enter side a: "))
b = float(input("Enter side b: "))
c = round(math.sqrt(a ** 2 + b ** 2), 2)
print(f"Hypotenuse: {c}")
```

10. Common Mistakes to Avoid

- **Using `^` for exponentiation:** `^` is bitwise XOR; use `**`.
- **Forgetting `import math`:** Raises a `NameError`.
- **Forgetting numeric conversion:** `input()` returns a string.
- **Confusing `round()`, `ceil()`, and `floor()`.**
- **Confusing units:** Circumference is cm, Area is cm², Volume is cm³.

12. Last-Minute Revision Sheet

1. Use `**` for power, not `^`.
2. Use `%` for remainders and even/odd parity checks.
3. Use augmented operators like `+=` and `*=`.
4. Always `import math` before using module functions.
5. Use `float(input(...))` for numeric user measurements.